

ME218C 2026 Communications Committee Presents:

Communications Protocol

I, a designated member of the ME218C Communications Committee—selected by a council of my peers—hereby acknowledge and agree to uphold the responsibilities outlined in the official Communications Protocol. I commit to adhering to this policy as the standard protocol for communication during the ME218C final project. I understand that my role is integral to fostering clarity, coordination, and accountability within my project team and the broader course community.

By printing my name below, I acknowledge that this written print serves as my official and approved signature for the purposes of this agreement:

Ege Turan
Alexander Chen
Ottavia Personeni
Gus Robinson
Yuhao Huai

Requirements for training a Quackcraft

Quackcrafts are prideful creatures, and will only respond to select commands. To set guidelines for the physical hardware implementation of the Mallard Module, the Communications Committee has defined the following components to be used after much Quackcraft observation:

Navigation

For nautical movement, the Quackcraft expects to receive **two differential values** coming from the Mallard Module. Each value represents a single axis (i.e. forward/backward or left/right), and is discretized (if analog) to fit in 8-bits for ease of transmission. Thus, a total of two bytes are allocated for planar motion. An example of a suitable input on the Mallard Module would be an analog XY joystick. **All processing for these signals will be internal within each Quackcraft;** this allows for creative input mechanisms.

Additional Buttons

In addition, each Quackcraft can expect **two bits of information from digital inputs**, which all controllers must implement. One bit shall be used for captain duck-smacking, while the other bit represents critter-collecting. Examples of captain duck-smacking would include a water gun or a prod, whereas critter collection could connect to actuating a gate. **Since many subspecies of Quackcraft exist, they do not need to perform any action upon receiving either bit if they do not have the functionality to do so** (if your Quackcraft is—for example—a peaceful manta ray, it would not need to do anything upon receiving a command to belligerently smack a captain duck, as manta rays are gentle creatures). Additionally, any team's Mallard Module may send up to six additional bits to their own Quackcraft, which will be detailed later. Each button will be implemented in a way that both completes the intended action and resets to the default state.

In the event of missed/**dropped packets** of data, Mallard Module operators will be expected to continue to try pressing the button again until the Quackcraft stops stubbornly ignoring commands as birds often do, and the expected action is completed. *(i.e. Team 1 has a button that opens their net when pressed. If the button is pressed and the net does not open, this might be an indication that the data packet is dropped. Team 1 will then wait briefly before pressing the button again, and repeat until they see their net open.)*

Pairing and Changing States

Finally, all Mallard Modules must implement a “**Quackcraft selector**” (for 5 Quackcrafts) and a “**Start Pairing**” input for pairing functionality. Details are left to each teams' discretion. Mallard Modules may include a switch to force the Quackcraft between navigation and refueling; any Mallard Modules without this functionality must perform this state change automatically.

Finding your runaway Quackraft

Due to their migratory, individualistic nature, Quackrafts are known to ignore their human compatriots within just a few seconds of separation. Hence, Mallard Module operators often find themselves needing to reconnect with runaway Quackrafts in order to ensure they can perform during showcase. Since each Quackraft will only respond to a single paired Mallard Module's commands, the following pairing procedure is essential.

Pairing

The Quackraft selector and Start Pairing input work in tandem to begin Quackraft communion. To initiate pairing, the Mallard Module's **Quackraft selector** is first dialed to the address of the desired Quackraft (Table 3). Notably, changing the Quackraft selector halts any preexisting communications being sent out by the Mallard Module. With the address selected, the **Start Pairing** input is pressed, which continuously sends out the default **Pairing** message to the Quackraft. Upon receiving a **Pairing** return-message with a charge status of 0xFF, the pairing ritual is complete.

During this time, if the target Quackraft is unpaired and receives the **Pairing** message, it will **store the address** of the associated Mallard Module and subsequently respond with a **Pairing** message containing a charge status set to 0xFF. The extroverted Quackraft, newly energized from forming a new bond, will be fully charged to 100% fuel. If the Quackraft is already paired with the Mallard Module that sent the pairing request, it will still relay a message with 0xFF charge, but such an interaction will not affect its current fuel level.

Obviously, paired Quackrafts—in an honorable display of loyalty—will not care to respond to any Mallard Module operators foolish enough to try to wrest control from another operator. (Mallard Module address information is stored in the Joy1 and Joy2 bytes for **pairing** messages). Teams must go through this pairing procedure, meaning you **cannot** send commands to a quackcraft you haven't received 0xFF back from. Quackrafts only need to indicate when they are first paired with a controller, and do not need to indicate which controller they are paired with or when they are unpaired. Any unpaired Quackraft should stop all movement and communication with previous controller.

Unpairing

If the Quackraft stops receiving messages from the paired Mallard Module for **4 seconds**, then it will coldly forget about said Mallard Module and will unpair of its own accord. Mallard Modules must keep sending packets to a Quackraft if they want to stay paired (a packet that says “don't shoot or move” still counts as a packet and thus they will remain paired). These should be transmitted at 5Hz class wide.

Mallard Modules can unpair from Quackrafts in two ways: (1) Powering off the Mallard Module; (2) Selecting a new Quackraft to pair to. Both options will stop all communication from the Mallard Module **to ANY Quackcraft**. To re-pair, the operator shall select the desired Quackcraft and begin the pairing process anew.

In the event of an unpair, the Quackraft and Mallard Module will return to their respective default states, as defined by each team.

Example: Sir Quackraft III has 50% fuel and receives no commands from its paired Mallard Module for 2 seconds. Then it gets a **pairing** message from that same Mallard Module. The Quackraft will respond with its own **pairing** message (0x02 as its status byte) with the current fuel level in the charge byte, but maintain fuel at 50%.

Example: Quackraft the Mighty has 50% fuel and receives no commands from its paired Mallard Module for 5 seconds. Then it gets a **pairing** message from that same Mallard Module. The Quackraft will respond with its own **pairing** message (0x02 as its status byte) with 240 in the charge byte, AND it will fully recharge to 100%.

Example: Karl Quackraft has 50% fuel and receives no commands from its paired Mallard Module for 2 seconds. Then it gets a **pairing** message from a different Mallard Module. Karl Quackraft will **NOT** respond to the new Mallard Module, as it is still looking for a message from its current pair. If 2 additional seconds pass with no efforts made to mend the current pair, Karl Quackraft will unpair on its own terms. Only then will it begin looking for new pairing partners.

Understanding your Quackraft's mood swings

Through much research into human-Quackraft communications, we have determined that the moody Quackraft has three primary states: **Pairing**, **Driving**, and **Charging**. These states should be saved in each Mallard Module so that operators can successfully communicate—after all, a Quackraft that is pairing or charging will steadfastly ignore any navigation commands. Additional states may be created by each team if desired.

Note that these **primary states should be stored in the Mallard Module**—the Quackraft is a complex, secretive creature—we can only wager a guess at what actually goes on inside. Still, storing its presumed fuel level is critical to ensuring we do not overwork it during showcase.

Driving

Driving the Quackraft is the primary state where we can command it to perform. This includes navigating across the arena, actuating any intake mechanisms, telling the Quackraft to attack captain-ducks, and any other team-specific functionality. While active in this mode, receiving any navigational or aggressive commands (such as moving around or smacking captain-ducks) will steadily drain fuel. Every Quackraft is genetically endowed with exactly **40 seconds** of fuel (i.e. when sending messages at 5 Hz, each actionable command will drain 0.2 seconds of fuel). Of course, performing both a navigational and an aggressive action in the same command will drain fuel at double the rate. Conversely, any idle Quackrafts are in perfect thermodynamic equilibrium—as superior beings, sitting ducks do not experience entropy and do not require fuel.

If a paired Quackraft runs out of fuel, the Mallard Module must send an idle command to the quackraft. This will have 127 (0x7F) in both the joystick bytes, representing no movement, and 0 in the button byte representing no actuation. This allows the Quackraft and Mallard Module to remain paired until recharging commences.

Charging

When the Mallard Module is in charging mode with all conditions met, it will send a **charging** message (status byte set to 0x01). When the Quackraft receives this **charging** message from its paired Module, it will **add 8 (so we fully charge in 5 full seconds)** to its total charge, up to a **maximum of 200 (chosen because of divisibility since 40 seconds x 5 Hz)**. If the charging conditions are not met for any reason, the Mallard Module will send **idle commands—driving** messages (status byte set to 0x00) with no thrust (0x7F) in the joystick bytes and 0 in the button byte—until the conditions are met again or the Mallard Module is switched out of charging mode.

The committee has decided that due to the nature of sending packets in charge mode, additional robustness will not be implemented—dropped packets of data will not be added thereafter.

Breaking down the human-Quackraft language barrier

As mentioned above, Quackrafts only respond to select commands, which must be sent out by paired Mallard Modules. This section dives into the semantics of Quackraft communication, showcasing the known range of commands that all Quackrafts understand and respond to.

Data Transmission

Data transmission will occur using the XBee API Frame mapped out in Pre-Lecture 49. The Mallard Module will transmit **13 byte messages from the Mallard Module to the Quackraft**, and the Quackraft will transmit **10 byte messages from the Quackraft to the Mallard Module**. Any Mallard Module should not exceed 5 Hz per message transmitted. Messages are sent at 9600 baud, 8N1. For more information, Section 3.4 of the [XBee Product Manual](#) provides an in-depth explanation for how data must be packaged for transmission/reception. Here are parameters agreed upon by the Communications Committee:

Mallard Module → Quackraft (13 bytes):

The following six bytes are required in every transmission to a Quackraft:

- Byte 1 is the **Start** delimiter, which is *always* 0x7E.
- Bytes 2-3 are the **Length** (# of bytes between **API ID** and the final **Data** byte, inclusive). This should *always* be 0x0009 for messages sent *by the Mallard Module*.
- Byte 4 is the **API ID**. For transmitting to a 16-bit address, this is always 0x01.
- Byte 5 is the **frame ID**. This labels the message between the PIC and XBee with a 1-byte identifier. Simply put, since messages are limited to 5 Hz, we don't need to disambiguate acknowledgements, so we set this to 0x00 in order to disable this labeling functionality.
- Byte 8 selects **Options**. Notably, setting this to 0x01 disables (ACK)nowledgement (which prevents the XBee from resending a dropped message up to three times).

Table 1: Required bytes for all Mallard Module → Quackraft messages

Start	Length		API ID	frame ID	Destination Address		Opt	Frame Data				Chk Sum
								Status	Joy1	Joy2	Digi	
0x7E	0x00	0x09	0x01	0x00	0x20	0x81-5	0x01					

The following seven bytes must be determined by each team to complete the transmission:

- Bytes 6-7 are the **Destination Address**, unique to each quackraft (Table 3). This tells the API which XBEE we are trying to send to/
- Bytes 9-12 are the **Frame Data**:
 - **Status** is either 0x00 (Driving), 0x01 (Charging), or 0x02 (Pairing).
 - **Joy1** maps the primary axis motion (likely forward/backward), with 0x00 as full throttle “reverse”, 0x7F being stationary, and 0xFF being full throttle “forward”.

- **Joy2** maps the secondary axis motion (likely left/right). Upon receiving a Pairing, with 0x00 being full throttle “left”, 0x7F being stationary, and 0xFF being full throttle “right”.
- **Digi** maps digital inputs. Bit 1 handles captain-duck smacking. Bit 0 handles critter collection. Bits 2-7 are free to be used at each team’s discretion, but will not be supported by every controller (not every Quackraft is fully compatible with every Mallard Module, after all!).
- Byte 13 is the **Checksum**. Adding this with all of the bytes after Length must equal 0xFF.

Note that for **Pairing** messages, the two bytes for Joy1 and Joy2 are repurposed to contain the Mallard Module’s address.

Table 2: Frame Data range of values:

	Driving	Charging	Pairing
Status	0x00	0x01	0x02
Joy1	(-) 0x00 ← 0x7F → 0xFF (+)	0x00	0x21
Joy2	(-) 0x00 ← 0x7F → 0xFF (+)	0x00	0x81-0x85
Digi	0x00 to 0x03*	0x00	0x00

**This range is supported on all Quackrafts, but the Digi value may range up to 0xFF*

Table 3: Team-specific Quackraft addresses

Team	1	2	3	4	5
Address	0x2081	0x2082	0x2087	0x2084	0x2086

To prevent cross-Quackcraft confusion, each Quackraft is assigned a specific address (Table 3). This goes in the destination address part of the API frame.

Table 4: Mallard Module → Quackraft Transmission Example Bytes

Start	Length		API ID	frame ID	Destination Address		Opt	Frame Data				Chk Sum
								Status	Joy1	Joy2	Digi	
Driving:												
0x7E	0x00	0x09	0x01	0x00	0x20	0x81	0x01	0x00	0xFF	0x7F	0x01	0xDD
Charging:												
0x7E	0x00	0x09	0x01	0x00	0x20	0x81	0x01	0x01	0x00	0x00	0x00	0x5B
Pairing:												
0x7E	0x00	0x09	0x01	0x00	0x20	0x81	0x01	0x02	0x21 *	0x81-5 *	0x00	0xB8

*Return address based on which controller is talking. MALLARD SOURCE ADDRESS this way the raft knows where to send its stuff back to

Table 4 above shows examples of how Team 1 can send **driving**, **charging**, and **pairing** messages:

- This **driving** message example would correlate with full throttle forward (0xFF), no lateral motion (0x7F), and actuating the critter collector (0x01) on a standard Quackraft.
- Within Frame Data, only the **Status** byte is nonzero in **charging** messages.
- **Pairing** messages encode the Mallard Module (source) address in the Joy1/Joy2 bytes. This lets the Quackraft know where to send its messages back in the destination address (detailed later).

Quackraft → Mallard Module (10 bytes):

Any well-trained Quackrafts will respond to Mallard Module commands with some form of acknowledgement. The following six bytes are seen in every transmission from a Quackraft:

- Byte 1 is the **Start** delimiter, which is *always* 0x7E.
- Bytes 2-3 are the **Length** (# of bytes between **API ID** and the final **Data** byte, inclusive). This should *always* be 0x0006 for messages sent *by the Quackraft*.
- Byte 4 is the **API ID**. For transmitting to a 16-bit address, this is always 0x01.
- Byte 5 is the **frame ID**. This labels the message between the PIC and XBee with a 1-byte identifier. Simply put, since messages are limited to 5 Hz, we don't need to disambiguate acknowledgements, so we set this to 0x00 in order to disable this labeling functionality.
- Byte 8 selects **Options**. Notably, setting this to 0x01 disables (ACK)nowledgement (which prevents the XBee from resending a dropped message up to three times).

Table 5: Required bytes for all Quackraft → Mallard Module messages

Start	Length		API ID	frame ID	Destination Address		Opt	Charge Byte	Chk Sum
0x7E	0x00	0x06	0x01	0x00	0x21	0x81-5	0x01	0x00	

The following seven bytes must be determined by each team to complete the transmission:

- Bytes 6-7 are the **Destination Address**, representing which Mallard Module the Quackraft is pairing with and trying to talk to. Address is delivered to the Quackraft during the pairing procedure. Table 6.
- Byte 9 is the **Charge Byte**:
 - **Charge** is used to convey the charge level of the boat during pairing, and to ensure no packets are dropped during the charging process.
- Byte 10 is the **Checksum**. Adding this with all of the bytes after Length must equal 0xFF.

Table 6: Team-specific Mallard Module addresses

Team	1	2	3	4	5
Address	0x2181	0x2182	0x2187	0x2184	0x2186